1.编写derainnet代码：

```
import torch as th
import torch.nn as nn

class ResBlock(nn.Module) :
    def __init__(self, in_channels, out_channels) :
        super(ResBlock, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, 1, 1, bias = False),
            nn.InstanceNorm2d(out_channels),
            nn.ReLU(inplace = True),
            nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias = False),
            nn.InstanceNorm2d(out_channels),
        )
        self.relu = nn.ReLU(inplace = False)

    def forward(self, x) :
        return self.relu(x + self.conv(x))

class DerainNet(nn.Module):
    def __init__(self, channels : int = 32, num : int = 4):
        super().__init__()
        self.entry = nn.Conv2d(3, channels, 3, 1, 1)
        self.pool = nn.MaxPool2d(2, 2)
        self.downs1 = nn.ModuleList()
        self.downs2 = nn.ModuleList()
        self.downs3 = nn.ModuleList()
        for i in range(num) :
            self.downs1.append(ResBlock(channels, channels))
        for i in range(num) :
            self.downs2.append(ResBlock(channels, channels))
        for i in range(num) :
            self.downs3.append(ResBlock(channels, channels))
        self.up2 = nn.ConvTranspose2d(channels, channels, 2, 2)
        self.up4 = nn.ConvTranspose2d(channels, channels, 4, 4)
        self.exit = nn.Sequential(
            nn.Conv2d(3 * channels, 3 * channels, 3, 1, 1),
            nn.InstanceNorm2d(3 * channels),
            nn.ReLU(inplace = True),
            nn.Conv2d(3 * channels, 3, 3, 1, 1)
        )

    def forward(self, x):
        f = self.entry(x)

        f1 = f
        for down in self.downs1:
            f1 = down(f1)
        f1 = f1 + f

        d2 = self.pool(f)
```

```
        f2 = d2
        for down in self.downs2 :
            f2 = down(f2)
        f2 = self.up2(f2 + d2)

        d3 = self.pool(d2)
        f3 = d3
        for down in self.downs3 :
            f3 = down(f3)
        f3 = self.up4(f3 + d3)

        fa = self.exit(th.cat([f1, f2, f3], dim = 1))

        return x - fa
```

2.训练结果：

```
(derain) pigeon@LAPTOP-GARN0974:~/ml-week/Image-Deraining-Recruitment-Task$ python main.py --data_dir "./data/Rain200L" --model derainnet --mode test --checkpoin
t "checkpoints/best_derainnet.pth.tar"
  warnings.warn(
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for
'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=AlexNet_Weights.IMAGENET1K_V1`. You
can also use `weights=AlexNet_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
Loading model from: /home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/lpips/weights/v0.1/alex.pth
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/lpips/lpips.py:107: FutureWarning: You are using `torch.load` with `weights_only=False` (the cur
rent default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_
only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via
 this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for
any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  self.load_state_dict(torch.load(model_path, map_location='cpu'), strict=False)
/home/pigeon/ml-week/Image-Deraining-Recruitment-Task/main.py:123: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value
), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flip
ped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless
they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where
 you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  load_checkpoint(torch.load(args.checkpoint, map_location=device), model, optimizer)
=> Loading checkpoint
EVAL => Avg PSNR: 33.5015 | Avg SSIM: 0.9439 | Avg LPIPS: 0.0302
```

3.

①下采样的定义是把特征图的尺寸变小，在代码里用池化来实现，作用是扩大感受野，通过降低分辨率来降低计算量；上采样是指把特征图的空间尺寸放大到原尺寸，在代码中通过转置卷积实现；concat是把两个空间尺寸相同的特征图按照通道维度拼接起来。

先下采样再上采样，是先用图像的分辨率来换感受野，再通过上采样把分辨率还原回去，同时通过多次下采样得到不同感受野的特征图，上采样后保证尺寸统一才能 `cat` 到一起。另外，先上采样再下采样还能减少中间计算的参数量。

②把多层特征通过 `cat` 拼接到一起，可以解决单一浅层或深层都不能表示整体、单一感受野无法兼顾不同雨丝尺度的情况。

附加1：还得自己找更好的方法

调研了PReNet和SPANet两篇文章，以下是结果：

1.对文章方法的理解和对我的启发：

PReNet的作者认为去雨一个渐进的过程，雨图中的雨本身就是分层的，因此期望用普通的残差网络做一次前向传播就完成去雨本身就是不合理的，这样做可能会使去雨不彻底，也可能会使图像本身的内容被抹去。

因此，PReNet认为去雨应该渐进地分阶段进行，在论文中去雨任务被PReNet分成了六个阶段，每个阶段的更新都是将上一个阶段的区域结果和原有雨图像按通道拼接起来成为一个六通道特征图，再将拼接后的特征图进行卷积并进入一个记忆单元，六个阶段完成后再经过残差块去雨预测出最终的无雨图。另外，PReNet在计算损失值时并不只是对最终的 x^T 计算损失，而是对中间过程中每个阶段的特征图都进行损失计算，这样既保证每个循环神经网络不会"摆烂"将工作留给后面的循环，又能通过将最终损失值的计算图连至浅层网络防止梯度消失。

PReNet的六个循环阶段共享同一套参数，这样跨阶段的权重共享避免了PReNet的参数量过大，更节省训练资源。

通过学习PReNet，虽然现在难以效仿PReNet实现多阶段的渐进去雨，但可以学习使用PReNet中通过参数共享控制参数大小的方法节省算力，也可以借鉴把阶段预测值和原始有雨图拼接在一起防止原始信息丢失的问题。另外，在训练深层网络时还可以利用多阶段监督来防止梯度消失的问题，使梯度更稳定。

SPANet认为雨层在有雨图中根本不是均匀分布的，而传统的网络在进行去雨的时候并不能处理这样的非均匀性，这样会导致没有雨的地方在去雨时被擦去图像内容，而雨较密集的区域则不能完全去雨。

为了解决这一问题，SPANet先用网络检测出哪些去雨需要去雨，再针对需要去雨的部分擦除，让无雨的部分保持不变。

在网络实现上，有雨图在SPANet网络上，先通过三个基础的残差块得到初始的浅层特征，再通过空间注意力残差块得到一张空间注意力图，再通过类似于赛博画师中局部风格迁移的方法将空间注意力图当作一个掩膜或者蒙板，与此同时将输入的特征图输进残差块中得到一个残差图，再把残差图乘上掩膜再加到原本的特征图上去，把这个过程重复四次最终得到无雨图。

为实现空间注意力图的提取，SPANet定义了空间注意力残差块，将空间注意力图的提取分为了两个分支，第一个分支为方向分支，输出四个方向的权重，分别对应上、下、左、右四个方向，生成四个方向的倾向度权重，用于判断雨层中雨丝的方向；第二个分支是主干提取分支，主干是通过进行四向的IRNN（使用单位初始矩阵、激活函数只用ReLU）搜集一个十字的信息，IRNN本来是循环神经网络，在这里我们将图像的"像素行"或"像素列"看作时间轴，这样我们就可以进行四向扫描，每次扫描类似于递推，令 $h_{i,j} = max(a_{dir}h_{i,j-1} + x_{i,j} + 1)$ ，这样下来需训练的参数只有衰减系数 a_{dir} ，这样这个分支就通过极小的参数量得到了雨丝的形状分析。两个分支得到的结果用cat拼接起来并再次进行一次四向IRNN的扫描，最终得出的结果经过卷积降维与激活就能得到最终的空间注意力图。

通过SPANet，我们可以学习在残差块中使用空洞残差块，在控制参数量不变的情况下扩大感受野。以及学习SPANet通过注意力图来解决图像雨层分布不均匀的问题。

2.这里只学习了题目给出的前两篇文章，没有自行寻找文章，故先不作答。

3.看到一篇文章，我会先用ai工具把文章翻译成中文，并于英文原文对照阅读，看到感觉不对的地方就去英文原文的地方找，阅读过程中先看摘要确定文章的目的以及解决的问题，再粗略看一遍模型的思想实现，遇到不理解的构造或者不知道的术语、实现等再去查询或者询问ai。

4.渐进式训练是指在训练过程中不使用一套恒定不变的设置，而是在不同阶段动态的改变训练的模式，在图像修复领域中渐进式训练一般表现为渐进式尺寸缩放，在前期裁剪小尺寸的图片进行训练，中期裁剪中尺寸的图片进行训练，后期用大尺寸的图片训练，这样做不仅能节省算力，也能优化损失曲面让损失曲面更平滑。

在未来科研中，渐进式训练自然是一种重要且有效的训练方法，但是也不能盲目使用，在使用前要根据训练目的等判断是否应该使用渐进式训练，例如在做创新模型架构的时候就应该控制好变量而不是使用渐进式训练。

5.在学习完两篇论文后，决定学习PReNet的渐进式递归和共享权重以及SPANet的空洞残差卷积和注意力门控，设计 `Mymodel` ，先将输入的有雨原图像与上一各阶段得到的图像拼接并进行一次卷积，再学习SPANet循环四次得到注意力图并进行出口卷积，循环这个操作三次，最终得到去雨图像。

①代码实现：

```
import torch as th
import torch.nn as nn

class Mask(nn.Module) :
    """
    思路：通过卷积+sigmoid激活函数生成注意力图
    作用：生成注意力图，作为掩膜“罩在”训练出来的残差上
    参考论文：SPANet
    """
    def __init__(self, in_channels : int):
        super().__init__()
        self.mask = nn.Sequential(
            nn.Conv2d(in_channels, in_channels // 4, kernel_size = 3, padding =
1),
            nn.ReLU(inplace = True),
            # 把所有通道压缩成一个，作为掩膜表示有雨的概率
            nn.Conv2d(in_channels // 4, 1, kernel_size = 1),
            nn.Sigmoid()
        )
    def forward(self, x) :
        return self.mask(x)

class SADBlock(nn.Module) :
    """
    思路：用空洞卷积扩大感受野，并采用生成的掩膜乘上得到的残差雨层解决内容被抹去的问题
    作用：残差卷积块，得到雨层
    参考论文：SPANet
    """
    def __init__(self, channels : int) :
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size = 3, padding = 1),
            nn.InstanceNorm2d(channels),
            nn.ReLU(inplace = True),
            nn.Conv2d(channels, channels, kernel_size = 3, padding = 2, dilation =
2),
            nn.InstanceNorm2d(channels),
        )
        self.mask = Mask(channels)
        self.relu = nn.ReLU(inplace = True)

    def forward(self, x) :
        rain = self.conv(x)
        mask = self.mask(x)
        return self.relu(x + rain * mask)
```

```
class Mymodel(nn.Module) :
    """
    思路：多阶段去雨，每个阶段把上一阶段的结果和初始有雨图cat在一起做残差网络，多次迭代得到去雨图
    作用：去雨
    参考论文：PReNet
    """
    def __init__(self, channels : int = 48, num_blocks : int = 4, iterations : int = 3) :
        super().__init__()
        self.channels = channels
        self.iterations = iterations

        self.entry = nn.Conv2d(6, channels, kernel_size = 3, padding = 1)
        self.blocks = nn.ModuleList()
        for _ in range(num_blocks) :
            self.blocks.append(SADBlock(channels))
        self.exit = nn.Conv2d(channels, 3, kernel_size = 3, padding = 1)

    def forward(self, x_input) :
        x = x_input
        for _ in range(self.iterations) :
            x_in = th.cat([x, x_input], dim = 1)
            feat = self.entry(x_in)
            for bolck in self.blocks :
                feat = bolck(feat)
            feat = self.exit(feat)
            x = x + feat
        return x
```

②计算模型参数的代码：

```
import os
import torch as th
from dataset import DerainDataset
from models.Mymodel import Mymodel
from torch.utils.data import DataLoader

def transform() :

  device = th.device("cpu")
  model = Mymodel().to(device)
  model.eval()
  n = sum(p.numel() for p in model.parameters())
  print(f"参数量：{n} | 模型大小：{(n * 4 / 1024 / 1024):.2f} MB")
  data_dir = "./data/Rain200L"
  ds = DerainDataset(data_dir, is_train = False)
  rainy, _, _ = next(iter(DataLoader(ds, batch_size = 1, shuffle = False)))

  path = "checkpoints/mymodel.onnx"
  os.makedirs("checkpoints", exist_ok = True)
```

```
    th.onnx.export(
      model, rainy, path,
      export_params = True,
      opset_version = 14,
      do_constant_folding = True,
      input_names = ["rainy_image"],
      output_names = ["derained_image"],
      dynamic_axes = {
        "rainy_image" : {0 : "batch", 2 : "H", 3 : "W"},
        "derained_image" : {0 : "batch", 2 : "H", 3 : "W"}
      },
    )

  if __name__ == "__main__" :
    transform()
```

运行结果： 参数量：191047 | 模型大小：0.73 MB ，模型结构被保存到/checkpoints/mymodel.onnx。

③④训练结果：

```
(derain) pigeon@LAPTOP-GARN0974:~/ml-week/Image-Deraining-Recruitment-Task$ python main.py --data_dir "./data/Rain200L" --model mymodel --mode test --checkpoint
"checkpoints/best_mymodel.pth.tar"
they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where
 you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  load_checkpoint(torch.load(args.checkpoint, map_location=device), model, optimizer)
=> Loading checkpoint
EVAL => Avg PSNR: 35.2263 | Avg SSIM: 0.9625 | Avg LPIPS: 0.0164
```

文中的去雨架构，如PReNet或SPANet，和这里的去雨任务的数据集大小不同，如果生搬硬套完全把网络架构搬过来只会浪费算力，且存在数据集过小欠拟合的情况；且一个去雨任务并不只由模型完成，如果不采用相应的数据处理、选择合适的优化器、调度器，或者没有进行合理的渐进式训练，那么就算使用先进的模型架构也不会有显著优秀的成功，因此更需要提炼先进架构的思想，在自己任务的基础上进行转化。